

Predict Once, Query Many: World Models that Forecast Conditional Trajectory Abstractions

Anonymous CVPR submission

Paper ID *****

Abstract

001 A world model lets a robot check candidate actions before
002 executing them. Latent world models such as DINO-WM
003 predict the features of every future frame and compare the
004 last one with a goal image, which is costly and a weak signal
005 of task progress. Direct scorers skip prediction, but must
006 learn what an action does separately for every goal. We
007 propose Conditional Trajectory Abstraction (CTA), a joint-
008 embedding world model that predicts a compact code of an
009 action's entire future segment, conditioned on the observed
010 history. During training, a target encoder learns this code
011 from real futures. A reader that never sees the action scores
012 the code against goal images, and a feature decoder keeps
013 the code a description of the future rather than a score. The
014 world model predicts the code from the history and a candi-
015 date action chunk in a single pass, so one prediction serves
016 any number of goals. Used to select among the proposals of
017 a frozen diffusion policy on PushT, CTA[main result].

018 1. Introduction

019 A world model lets a robot check an action before taking
020 it. Latent world models for visual control, such as DINO-
021 WM [37] and V-JEPA 2-AC [3], do so by predicting the
022 features of every future frame and measuring how close the
023 last one is to a goal image. For choosing between candidate
024 actions, this is both expensive and uninformative. It is expen-
025 sive because every candidate requires predicting hundreds of
026 patch features per step, most of which repeat what the robot
027 already sees. It is uninformative because feature distance
028 to a goal is a weak signal of task progress. On PushT, even
029 when computed on the *true* future frames, it barely separates
030 the good proposals of a diffusion policy from the bad ones:
031 it improves success by 3 points, while a learned ranking of
032 the same true futures improves it by 8.5.

033 The opposite extreme skips prediction. A direct scorer
034 maps the observation, a candidate action, and the goal
035 straight to a score [29]. It is cheap, but it entangles what the

action does with how the outcome relates to the goal, and
must learn the former anew for every goal.

We argue that a world model for decision making should
predict neither every frame nor nothing, but a compact de-
scription of what the action will do: only as much as the deci-
sion needs, and only what the history does not already reveal.
The second part is the principle of conditional video cod-
ing [25]: when encoder and decoder share the past frames,
bits are spent only on what is new. We apply it to the future
of an action that has not been executed yet.

We call the resulting model *Conditional Trajectory Ab-
straction* (CTA, Fig. 1). During training, a target encoder
compresses the actual future of an action chunk into 16 dis-
crete tokens, conditioned on the history. A reader scores the
code against a goal image, and a feature decoder reconstructs
the future from it. Neither sees the action, so everything the
action changes must pass through the code. The world model
learns to predict the code from the history and the action in
a single forward pass, as the predictor of a joint-embedding
architecture [2, 24]. At test time, a frozen policy proposes
candidate chunks, the world model predicts one code per
candidate, and the reader ranks them. Because codes do not
depend on the goal, one prediction serves any number of
goals.

Our contributions are:

- a formulation of a world model's target as a conditional
code of an action's future segment, with per-step latent
world models and direct scorers as its two limiting cases;
- CTA, a joint-embedding world model that learns this code
with a ranking reader and a conditional feature decoder,
and predicts it in one pass;
- an attribution ladder that separates what is lost by com-
pressing the future from what is lost by predicting it;
- experiments on PushT with a released diffusion policy,
where CTA[result vs. policy, direct scorer, endpoint and
per-step latent world models, with latency]. [TODO: Sec-
ond task in which progress depends on the path.]

Deployment: predict once, query many

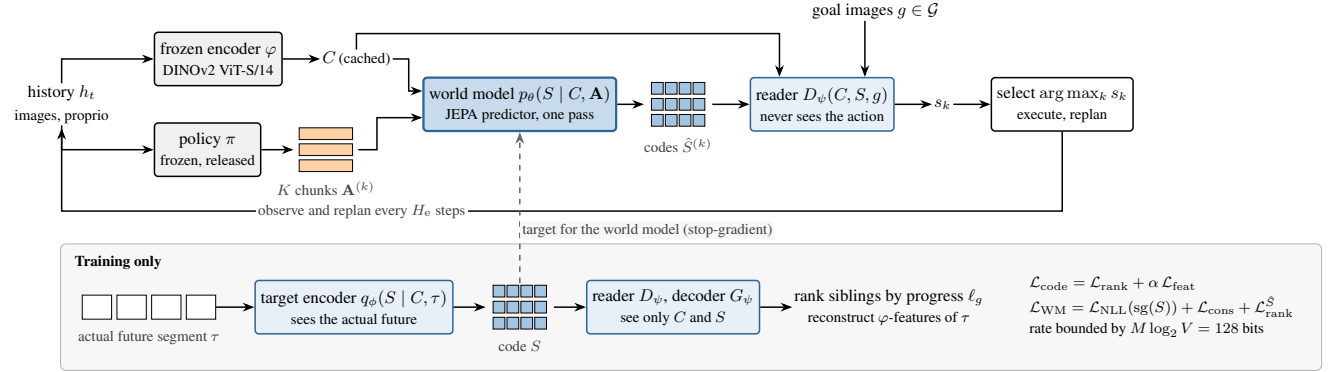


Figure 1. **Conditional Trajectory Abstraction.** *Top:* at each decision, the history is encoded once into a context C of frozen DINOv2 features, and a frozen policy proposes K action chunks. In one batched pass, the world model predicts for each chunk a 16-token code $\hat{S}$ of the chunk’s entire future segment, conditioned on C . A reader that sees C and the code, but never the action, scores each code against goal images; one prediction serves any number of goals. *Bottom:* during training, a target encoder compresses the actual future segment into the code given C . The reader ranks sibling candidates by progress, and a decoder reconstructs the future’s frozen features from (C, S) . The world model predicts the stop-gradient code and is also trained through the frozen reader.

2. Related Work

Latent and joint-embedding world models. Latent world models learn dynamics in a compact state space and plan or learn behaviors in imagination [17–19]; token-based models predict discrete image tokens autoregressively [27]. Δ -IRIS encodes only the stochastic change between consecutive frames given the past [28], the closest per-step relative of conditional coding. JEPAs predict the representation of a target from that of its context instead of reconstructing it [2, 5, 24]. For control, DINO-WM [37] and V-JEPA 2-AC [3] predict the patch features of a frozen encoder [30] step by step and plan by feature distance to a goal image. CompACT [21] compresses each observation into a few tokens to speed up such planning, and GCQ [31] quantizes observation–action sequences with an action-conditioned codebook. In all of these, the prediction target is fixed in advance: an observation, or a latent of it, at each time step. CTA keeps the JEPA recipe but learns the target: one code per action segment, conditioned on history that the reader also sees.

Temporally extended prediction. Option models [34] and trajectory-level dynamics models [10, 23] predict the outcome of an extended action without stepping through every intermediate state, and jumpy models compose such predictions across timescales [13]. Successor features and their action-sequence variants predict cumulative features [6, 7], and TD-Flow models discounted future-state occupancy [12]. These summarize the future as a state at one time or as an order-free aggregate, predicted from scratch. CTA codes the whole segment, including its intermediate frames, and spends its bits only on what the history does not determine.

Policy steering with world models. A frozen generative

policy can be improved at deployment by sampling proposals and selecting one with an external scorer. GPC [32], the closest system to ours, ranks diffusion-policy proposals with a reward model applied to futures predicted by an action-conditioned world model. FOREWARN [35], Dream-Steer [9], and DynaGuide [11] steer or rank with vision–language, value, or dynamics models on latent predictions, and V-GPS and RoboMonkey rank proposals without a world model [22, 29]. We adopt the same propose-then-select interface and, like GPC and V-GPS, train the scorer with task-progress labels. Our difference is what the world model predicts: a goal-independent code that one reader evaluates for any goal image. Our endpoint and per-step latent baselines follow the GPC recipe in latent space.

Conditional coding and decision-aware compression. Learned codecs optimize a rate–distortion trade-off end to end [4], and DCVC [25] conditions both encoder and decoder on the temporal context. Unlike the Wyner–Ziv setting [36], where only the decoder has side information, our encoder and heads both see the context. Value-equivalent models [1, 16, 33] keep only what is needed to predict values. CTA differs in two ways: its source is the unexecuted future of an action, which must be predicted before execution, and its distortion combines a decision term with a generic reconstruction term so that the code is not specialized to the training goals.

3. Problem Formulation

Setting. At decision time t , a frozen encoder φ maps the robot’s images to patch features, and the context C_t collects the features of the current and previous images together with recent proprioception. A candidate chunk

$\mathbf{A} = (a_t, \dots, a_{t+H_e-1})$ would produce a *future segment* τ : the features of the images it generates, subsampled in time, and the final proprioception. The world model must describe τ before $\mathbf{A}$ is executed.

Selecting among policy proposals. At each decision, a frozen pretrained policy π proposes K chunks, $\mathbf{A}^{(1)}$ being the one it would execute on its own. A scorer ranks them, the best is executed (ties keep $\mathbf{A}^{(1)}$), and the policy replans [9, 32]. Selecting among a competent policy’s proposals, rather than optimizing actions against the model, keeps candidates where a learned model is reliable [15, 20]. All scorers rank the same proposals.

Goals and labels. Goals are given as images $g \in \mathcal{G}$. During training, the simulator labels each candidate with its task progress ℓ at the end of the segment; on PushT, this is the negative mean distance between the block’s vertices and their target positions, as in GPC [32]. Candidates share a starting state, so only their order matters. No label is used at test time.

Conditional trajectory code. A target encoder $q_\phi(S | C, \tau)$, used only in training, maps the actual future to M discrete tokens $S \in \{1, \dots, V\}^M$. Two heads read the code with the context but never the action: a reader $D_\psi(C, S, g)$ that scores progress toward g , and a feature decoder $G_\psi(C, S)$ that reconstructs τ . A world model $p_\theta(S | C, \mathbf{A})$ predicts the code before execution. The code solves a conditional compression problem,

$$\begin{aligned} \min_{q_\phi, \psi} \mathbb{E}[d_{\text{rank}}(D_\psi(C, S, g), \ell) + \alpha d_{\text{feat}}(G_\psi(C, S), \tau)] \\ \text{s.t. } I(\tau; S | C) \leq R, \end{aligned} \quad (1)$$

where both the encoder and the heads see the context. Since S is a deterministic function of (C, τ) , the code length bounds the rate: $I(\tau; S | C) = H(S | C) \leq M \log_2 V$, or 128 bits for $M=16$ and $V=256$. Only the part of the code that the action determines, $I(S; \mathbf{A} | C)$, can be forecast; we estimate it in Sec. 4.4.

Two limiting cases. A *per-step latent world model* such as DINO-WM sets $S = \tau$: it predicts every patch of every step, without compression. A *direct scorer* $D(C, \mathbf{A}, g)$ removes S : prediction and evaluation are fused, so every candidate-goal pair costs a full evaluation. CTA sits between them at an explicit rate (Tab. 1).

4. Conditional Trajectory Abstraction

CTA has three learned parts: a target encoder that turns the actual future into a code (Sec. 4.2), two heads that read the code (Sec. 4.3), and a world model that predicts the code from an action (Sec. 4.4).

	Goal-independent prediction	Per candidate and goal	Sequential passes
Per-step latent WM	$H_e \cdot P$ tokens	readout	H_e
Endpoint latent WM	P tokens	readout	1
Direct scorer	none	full model	1
CTA (ours)	M tokens	reader	1

Table 1. **Design properties** of models that evaluate a candidate chunk ($P=256$ patch tokens per image, $M=16$ code tokens). A direct scorer has no goal-independent part, so its whole cost recurs for every goal.

4.1. Inputs

As in DINO-WM [37], images are embedded by a frozen DINOv2 ViT-S/14 [30]; we project the patch features to 128 dimensions with PCA. The context C contains the 16×16 patch tokens of the current image, the previous image pooled to 8×8 , and the proprioception at both times. The future segment τ contains the full token grid of the final image, pooled tokens of intermediate images with temporal embeddings, and the final proprioception. Features are computed once per image and shared by all candidates and goals.

4.2. Target encoder

The target encoder $q_\phi(S | C, \tau)$ is used only during training. M learned queries cross-attend to the future and context tokens, and each output is quantized by FSQ [26] with levels (8, 8, 4), giving 256 values per token. Because the queries see the context, the encoder learns which parts of the future the history already implies and spends the code on the rest. The M tokens describe the segment as a whole, not one frame each.

4.3. Reader and feature decoder

The reader $D_\psi(C, S, g)$ is a Transformer over the context tokens, the code tokens, and the patch tokens of a goal image, and outputs one score. It is trained on candidates that branch from the same state, with a pairwise logistic loss over pairs whose labels differ by more than a margin ϵ :

$$\begin{aligned} \mathcal{L}_{\text{rank}} = \mathbb{E}_{(i,j): \ell_i > \ell_j + \epsilon} [\log(1 + e^{-(s_i - s_j)})], \\ s_i = D_\psi(C, S_i, g). \end{aligned} \quad (2)$$

Ranking within a decision matches how the score is used, so it need not be calibrated across states.

A ranking loss alone would let the code shrink to whatever orders the training goals. The feature decoder $G_\psi(C, S)$ prevents this by reconstructing the future tokens from the context and the code. It predicts each token’s change from the current image, and its loss $\mathcal{L}_{\text{feat}}$ is normalized by that of predicting no change, so an uninformative code scores one. In JEPA terms, the decoder keeps the target representation from collapsing [24]; no image is ever generated.

4.4. World model

The world model $p_\theta(S \mid C, \mathbf{A})$ is the JEPA predictor. A Transformer encoder reads the context tokens and one token per action, and M learned queries decode all code tokens in parallel. Each FSQ coordinate of each token is predicted as a categorical distribution over its levels, and the reader receives the expected codeword $\hat{S}$. A single forward pass therefore scores the whole set of proposals. The world model never sees future images, the goal, or labels at test time.

It is trained against the stop-gradient code $S = \text{sg}(q_\phi(C, \tau))$:

$$\mathcal{L}_{\text{WM}} = \mathcal{L}_{\text{NLL}} + \mathcal{L}_{\text{cons}} + \mathcal{L}_{\text{rank}}^{\hat{S}}. \quad (3)$$

$\mathcal{L}_{\text{NLL}}$ is the cross-entropy of each coordinate of S . The other two terms pass through the frozen reader: $\mathcal{L}_{\text{cons}}$ matches the within-decision score differences of $\hat{S}$ to those of S , and $\mathcal{L}_{\text{rank}}^{\hat{S}}$ applies (2) to the predicted codes, weighting pairs by their label difference. A context-only model $r_\omega(S \mid C)$ is trained with $\mathcal{L}_{\text{NLL}}$ alongside; its excess cross-entropy over the world model estimates $I(S; \mathbf{A} \mid C)$, the part of the code the action explains.

4.5. Training

Branched data. At every decision of a policy rollout, we execute each proposal for H_e steps from a clone of the simulator state, obtaining several futures and labels from one history. To make siblings differ enough to rank, we also branch a copy of each proposal shifted by a constant offset. In half of the rollouts, half of the decisions follow the best branch rather than the default one, so that the data cover the states a good selector visits. All branches of an episode stay in one split.

Stages. Stage 1 trains the target encoder, reader, and decoder on actual futures with $\mathcal{L}_{\text{code}} = \mathcal{L}_{\text{rank}} + \alpha \mathcal{L}_{\text{feat}}$, using straight-through gradients through the quantizer. Stage 2 freezes them and trains the world model with (3); each network keeps the checkpoint with the best development objective. An optional stage co-designs the code for predictability, alternating code and world-model updates with a penalty $\lambda \|S - \mathbb{E}_{\bar{p}_\theta}[S]\|^2$ under a frozen copy of the world model; we treat it as an ablation. Throughout, we monitor code perplexity, distinct codes per decision, the decoder's R^2 , and $I(S; \mathbf{A} \mid C)$ to detect collapse.

4.6. Planning

Algorithm 1 summarizes deployment. Per decision, CTA encodes the images once, runs the world model once on all proposals, and runs the reader on every candidate-goal pair; a new goal costs only reader evaluations.

4.7. Implementation details

All modules have width 256 and 8 heads: the target encoder has three decoder layers, the reader four encoder layers, the

Algorithm 1 Selecting policy proposals with CTA.

Require: frozen policy π and encoder φ ; world model p_θ ; reader D_ψ ; goal images $\mathcal{G}$

- 1: **while** episode not finished **do**
- 2: $C \leftarrow \varphi(h_t)$ ▷ once per decision
- 3: $\{\mathbf{A}^{(k)}\}_{k=1}^K \leftarrow$ proposals of π
- 4: $\hat{S}^{(k)} \leftarrow$ expected code of $p_\theta(\cdot \mid C, \mathbf{A}^{(k)})$ ▷ one batched pass
- 5: $s_k \leftarrow \frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} D_\psi(C, \hat{S}^{(k)}, g)$
- 6: $k^* \leftarrow$ smallest index attaining $\max_k s_k$ ▷ ties keep $\mathbf{A}^{(1)}$
- 7: execute $\mathbf{A}_{1:H_e}^{(k^*)}$; $t \leftarrow t + H_e$

feature decoder two decoder layers, and the world model four encoder and four decoder layers. We use $M=16$ (128 bits), $\alpha=1$, and $\epsilon=10^{-3}$. Stage 2 runs 8,000 AdamW updates (learning rate 10^{-4} , weight decay 0.05, bfloat16) with 32 decisions per update. **[TODO: Final stage-1 schedule.]** On PushT, 96×96 images are resized to 224×224 for DINOv2, $H_e=8$, and the intermediate images follow steps 2, 4, and 6. Each action is a target position of the agent, embedded in absolute and relative coordinates.

5. Experiments

We use CTA to select among the proposals of a frozen diffusion policy and compare it with latent world models and a direct scorer that rank the same proposals. We then locate where ranking quality is lost and ablate the design.

5.1. Setup

Task and policy. On PushT [14], an episode succeeds when the block covers over 95% of the target within 300 steps. We use the released LeRobot Diffusion Policy [8], which executes 8 of 16 denoised actions; unmodified, it succeeds on 65 of 100 episodes (65.4% reported). At each decision it proposes $K=8$ chunks. **[TODO: Lock whether the bank also holds a perturbed copy of each proposal.]**

Data. We train on 2,000 branched rollouts (Sec. 4.5), 87,860 decisions in total. Separate episodes are used for development (300) and for a sealed test set (400) that is evaluated once, with three training seeds. Goal images are the final frames of 16 successful episodes outside all splits.

Baselines. The *direct scorer* $D(C, \mathbf{A}, g)$ is trained with the same data and loss as CTA. The *endpoint* and *per-step latent world models* predict the DINOv2 features of the final frame or of every frame, in the manner of DINO-WM, and are scored by a reader trained on real futures. **[TODO: Implement the per-step latent world model; retrain the endpoint model on the same data.]** We also report three references that simulate every candidate and are therefore not deployable: the *oracle*, which picks the best label; a *reader on the true future*; and *feature distance on the true*

	Success (%)	ms / decision
Policy only	—	—
Direct scorer	—	—
Endpoint latent WM	—	—
Per-step latent WM	—	—
CTA (ours)	—	—
<i>Simulates every candidate (not deployable)</i>		
Feature distance, true future	—	—
Reader, true future	—	—
Oracle	—	—

Table 2. **Closed-loop success on PushT** (sealed test set, three training seeds) and planning time per decision, including proposal sampling. All methods select among the same proposals.

Reader input	Retained gap
True future, uncompressed	—
Code of the true future	—
Predicted code (CTA)	—
Predicted code, NLL-only WM	—
Predicted final-frame features	—
Actions only (direct scorer)	—

Table 3. **Attribution ladder** on development decisions (episode-clustered 95% intervals). The drop from row 1 to 2 is lost to compression; from row 2 to 3, to prediction.

future, DINO-WM’s planning cost under perfect prediction. **Metrics.** We report closed-loop success, with episode-paired bootstrap intervals and exact McNemar tests, and planning time per decision. Offline, the *retained gap* is the fraction of the oracle’s gain over $A^{(1)}$ that a scorer’s choices recover.

5.2. Main results

[Main result: CTA vs. policy only and vs. the direct scorer; fraction of the oracle’s and the true-future reader’s gain recovered; time relative to the latent world models.]

5.3. Where is ranking lost?

Table 3 scores the same decisions as the reader’s input moves from the true future to its code to the code predicted from actions. [Interpretation, with code diagnostics: perplexity, distinct codes per decision, $I(S; A | C)$ in bits, feature R^2 .]

5.4. Ablations

Each ablation changes one choice and keeps the data and training budget fixed: the target encoder without the history, or with only the final frame; a per-step code with the same total bits, which would refute the segment design if it matched CTA; no feature decoder ($\alpha=0$); a world model trained with $\mathcal{L}_{NLL}$ only; co-design with $\lambda>0$; and code lengths $M \in \{8, 16, 32\}$. [Ablation table.]

5.5. When progress depends on the path

[TODO: Second task whose success depends on what happens inside a segment. Key contrast: CTA vs. the final-frame code at equal bits, and vs. the per-step latent world model at equal quality.]

6. Conclusion and Limitations

[Conclusion, written after results.]

Limitations. CTA is not label-free: task-progress labels from the simulator train the reader and the ranking term of the world model, although nothing uses labels at test time. The proposal policy is pretrained on expert demonstrations. Training data require branching the simulator at every decision. A selector cannot exceed the best candidate in the bank, so gains are bounded by the policy’s proposals.

References

- [1] Dilip Arumugam and Benjamin Van Roy. Deciding what to model: Value-equivalent sampling for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 2
- [2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 1, 2
- [3] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 1, 2
- [4] Johannes Ballé, Valero Laparra, and Eero P. Simoncelli. End-to-end optimized image compression. In *International Conference on Learning Representations (ICLR)*, 2017. 2
- [5] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research (TMLR)*, 2024. 2
- [6] André Barreto, Will Dabney, Rémi Munos, Jonathan J. Hunt, Tom Schaul, Hado van Hasselt, and David Silver. Successor features for transfer in reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2
- [7] Boyuan Chen, Chuning Zhu, Pulkit Agrawal, Kaiqing Zhang, and Abhishek Gupta. Self-supervised reinforcement learning that transfers using random features. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [8] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. 4
- [9] Hanchen Cui et al. DreamSteer: Latent world models can steer VLA policies during deployment without any finetuning. *arXiv preprint arXiv:2607.02865*, 2026. 2, 3

- [10] Zihan Ding, Amy Zhang, Yuandong Tian, and Qingqing Zheng. Diffusion world model: Future modeling beyond step-by-step rollout for offline reinforcement learning. *arXiv preprint arXiv:2402.03570*, 2024. 2
- [11] Maximilian Du and Shuran Song. DynaGuide: Steering diffusion policies with active dynamic guidance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [12] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Rémi Munos, Alessandro Lazaric, and Ahmed Touati. Temporal difference flows. In *International Conference on Machine Learning (ICML)*, 2025. 2
- [13] Jesse Farebrother, Matteo Pirodda, Andrea Tirinzoni, Marc G. Bellemare, Alessandro Lazaric, and Ahmed Touati. Compositional planning with jumpy world models. *arXiv preprint arXiv:2602.19634*, 2026. 2
- [14] Pete Florence, Corey Lynch, Andy Zeng, Oscar Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2021. 4
- [15] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. 3
- [16] Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 2
- [17] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019. 2
- [18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.
- [19] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. 2
- [20] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. 3
- [21] Dongwon Kim, Gawon Seo, Jinsung Lee, Minsu Cho, and Suha Kwak. Planning in 8 tokens: A compact discrete tokenizer for latent world model. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 2
- [22] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. In *Conference on Robot Learning (CoRL)*, 2025. 2
- [23] Nathan O. Lambert, Albert Wilcox, Howard Zhang, Kristofer S. J. Pister, and Roberto Calandra. Learning accurate long-term dynamics for model-based reinforcement learning. In *IEEE Conference on Decision and Control (CDC)*, 2021. 2
- [24] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, version 0.9.2, 2022. 1, 2, 3
- [25] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 1, 2
- [26] Fabian Mentzer, David Minnen, Eiríkur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations (ICLR)*, 2024. 3
- [27] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023. 2
- [28] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning (ICML)*, 2024. 2
- [29] Mitsuhiro Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. In *Conference on Robot Learning (CoRL)*, 2024. 1, 2
- [30] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024. 2, 3
- [31] Xiangyuan Peng, Xingsi Dong, and Si Wu. Vector quantization in the brain: Grid-like codes in world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. 2
- [32] Han Qi, Haocheng Yin, Aris Zhu, Yilun Du, and Heng Yang. Inference-time enhancement of generative robot policies via predictive world modeling. *IEEE Robotics and Automation Letters (RA-L)*, 2025. 2, 3
- [33] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 2
- [34] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. 2
- [35] Yilin Wu, Ran Tian, Gokul Swamy, and Andrea Bajcsy. From foresight to forethought: VLM-in-the-loop policy steering via latent alignment. In *Robotics: Science and Systems (RSS)*, 2025. 2
- [36] Aaron D. Wyner and Jacob Ziv. The rate-distortion function for source coding with side information at the decoder. *IEEE Transactions on Information Theory*, 22(1):1–10, 1976. 2
- [37] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning (ICML)*, 2025. 1, 2, 3